

BE Automatique

Prune Mamalet

Garance Werner-Guyader

December 2024

1 Objectif de l'étude

On souhaite modéliser le comportement du robot légo afin d'implémenter ensuite le code nécessaire en C pour qu'il se maintienne seul sur sa position d'équilibre.

Pour se faire, on assimile ce robot à un pendule inversé dont on simule le comportement sur simulink sous différentes contraintes.

1.1 Modèle mathématique

$$(S) \begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = \frac{g}{l} \sin(x_1(t)) - \frac{\cos(x_1(t))u(t)}{l} \\ x_1(0) = x_{0,1} = \alpha_0 \\ x_2(0) = x_{0,2} = \dot{\alpha}_0, \end{cases}$$

Figure 1:

Le système n'étant pas linéaire du fait de l'utilisation des fonctions trigonométriques, il pourra se stabiliser uniquement autour de ses points de fonctionnement (x_e, u_e) .

On pose :

$$f: \mathbb{R}^3 \rightarrow \mathbb{R}^2$$

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} x_2 \\ \frac{g}{l} \sin(x_1) - \frac{\cos(x_1) * u}{l} \end{bmatrix}$$

Et après calcul (on résout $f(x_e, u_e) = 0$) on obtient : $(x_e, u_e) = (x_{e1}, 0, g * \tan(x_{e1}))$ avec $x_{e1} \in \mathbb{R} \setminus \left\{ \frac{\pi}{2} + k\pi \mid k \in \mathbb{Z} \right\}$

Enfin, puisque $u(t) = u_e + K^*(x(t) - x_e)$, on a $f(x, u) = f(x, u_e + K^*(x(t) - x_e)) = g(x)$.

Et alors, en théorie, le système stabilise autour de (x_e, u_e) si et seulement si la matrice jacobienne de g évaluée en x_e est à valeurs propres à partie réelle strictement négative.

Après calcul, on a:

$$\begin{cases} \det(J_g) > 0 \\ \text{tr}(J_g) < 0 \end{cases} \iff \begin{cases} k1 > g + \tan(x - e1) * u_e \\ k2 > 0 \end{cases}$$

Et on nous donne $(x_e, u_e) = (0, 0, 0)$ donc $\mathbf{K} = \begin{bmatrix} k1 > g \\ k2 > 0 \end{bmatrix}$

2 TP2 : Simulation du pendule inversé contrôlé

En pratique, la stabilité du système dépend de d'autres facteurs et c'est ce qu'on va observer en faisant varier x_0 , K et l'intégrateur dans cette partie.

2.1 Contrôle par retour d'état

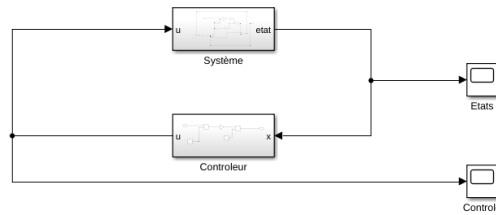


Figure 2:

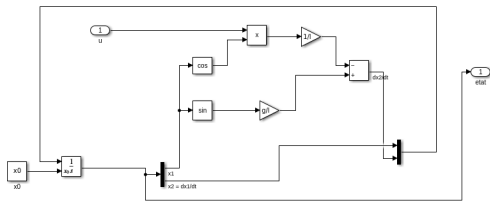


Figure 3: Système

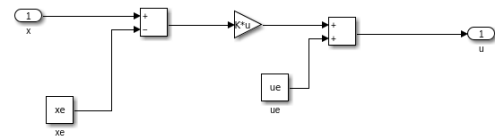


Figure 4: Contrôleur

Données à tester :

Cas	x_0	t_f	K	Intégrateur
Cas 1.1	$(\pi/20, 0)$	10	$(30, 10)$	ode45
Cas 1.2	$(\pi/20, 0)$	100	$(10, 1)$	ode45
Cas 1.3	$(\pi/20, 0)$	100	$(10, 1)$	Euler, ode1
Cas 1.4	$(\pi/20, 0)$	1000	$(10, 1)$	Euler, ode1
Cas 1.5	$(\pi/10, 0)$	100	$(10, 1)$	ode45
Cas 1.6	$(\pi/10, 0)$	100	$(30, 10)$	ode45

Figure 5: Tableau des données

- Cas 1.1

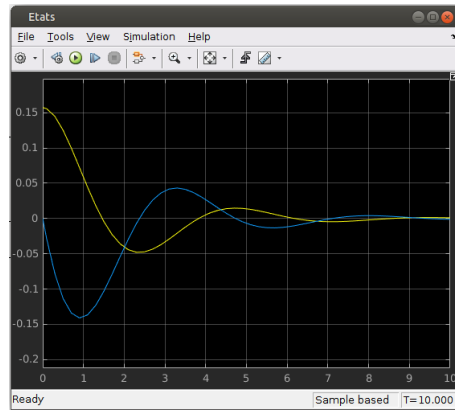


Figure 6: Simulation état Cas1.1

- Cas 1.2

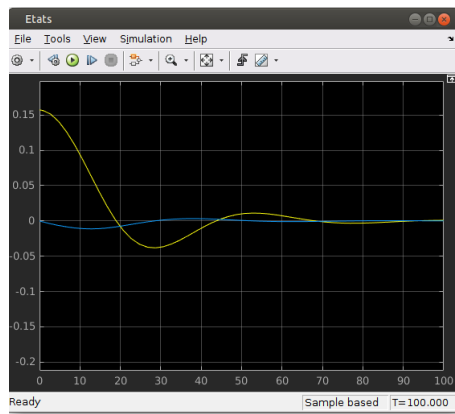


Figure 7: Simulation état Cas1.2

- Cas 1.3

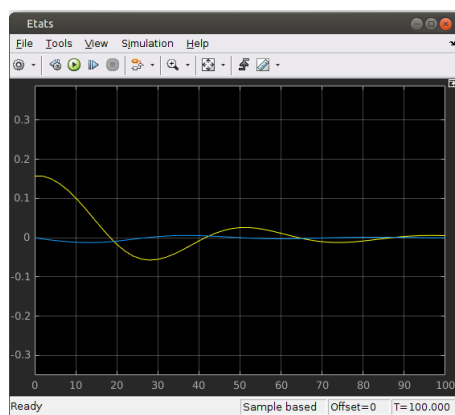


Figure 8: Simulation état Cas1.3

- Cas 1.4

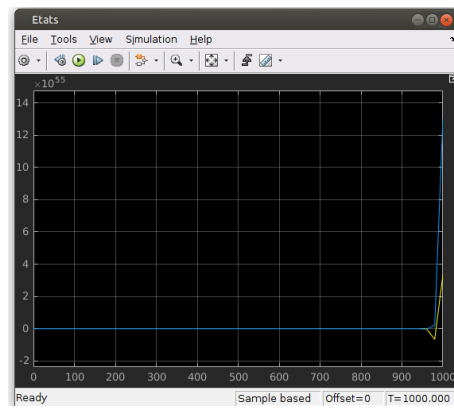


Figure 9: Simulation état Cas1.4

- Cas 1.5

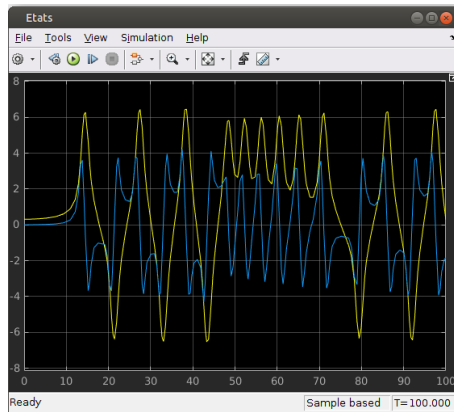


Figure 10: Simulation état Cas1.5

- Cas 1.6

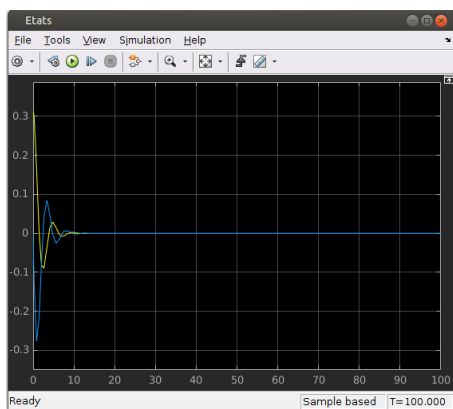


Figure 11: Simulation état Cas1.6

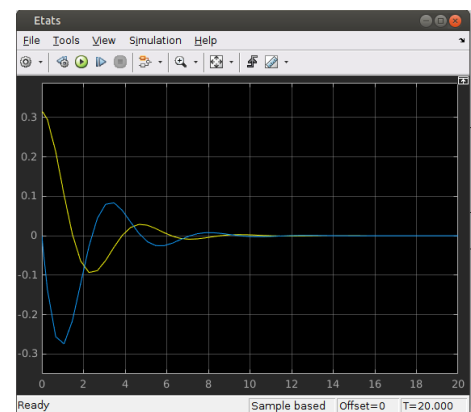


Figure 12: Simulation état Cas1.6 avec $t_f = 20$ pour plus de lisibilité

Analyse des resultats:

- En comparant les cas 1.3 et 1.4, c'est à dire en gardant des paramètres identiques mais en lançant une simulation dix fois plus longue, on observe que les courbes divergent. On se rend compte que même si les courbes convergent vers la position d'équilibre en un temps très court, à l'infini le pendule semble perdre sa position d'équilibre
- En comparant les cas 1.2 et 1.5, cas pour lesquels seule la position d'origine x_0 est différente, on remarque que plus elle est éloignée du point d'équilibre, plus les courbes divergent et donc moins le système a de chances de se stabiliser de nouveau autour de ce point.
- En comparant les cas 1.1 et 1.2, on remarque que plus la valeur du gain K est importante, plus les courbes convergent vite, ie plus le système retrouve vite sa position d'équilibre.

2.2 Controle par retour d'état avec capteur et prédicteur

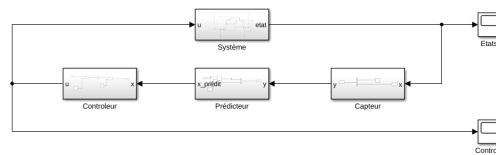


Figure 13:

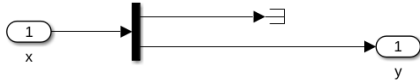


Figure 14: Capteur



Figure 15: Prédicteur

Données à tester :

Cas	x_0	t_f	K	pas	Intégrateur
Cas 1	$(\pi/20, 0)$	100	(10, 1)	par défaut	ode45
Cas 2	$(\pi/20, 0)$	100	(10, 1)	0.001	Euler, ode1
Cas 3	$(\pi/20, 0)$	100	(10, 1)	5	Euler, ode1

Figure 16: Tableau des données

- Cas 1

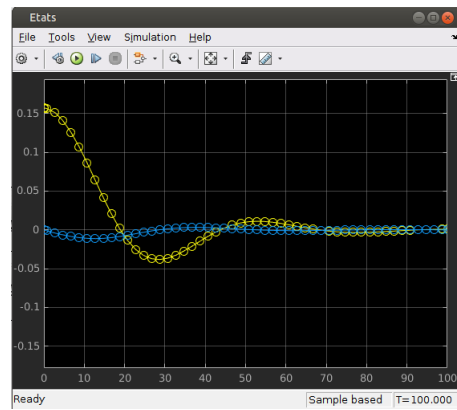


Figure 17: Simulation état Cas 1

- Cas 2

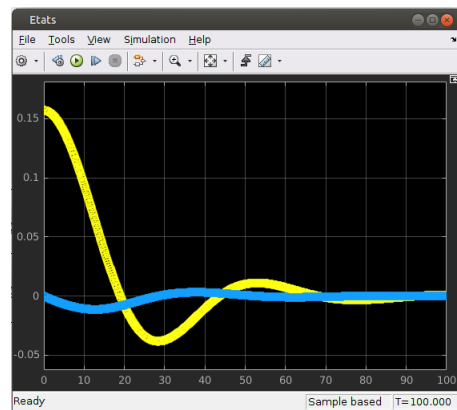


Figure 18: Simulation état Cas 2

- Cas 3

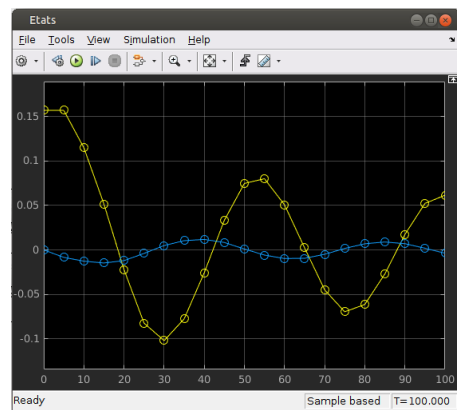


Figure 19: Simulation état Cas 3

Analyse des resultats:

- En comparant les cas 2 et 3, on remarque que plus le pas est grand moins les courbes convergent vite (pour pas=5 ce n'est même pas visible sur le graphe).

- En comparant les cas 1 et 2, on remarque que choisir un pas faible et un integrateur d'Euler revient au même que choisir un intégrateur Ode 45.

3 TP3 : Simulation du robot logo

3.1 Modèle continu

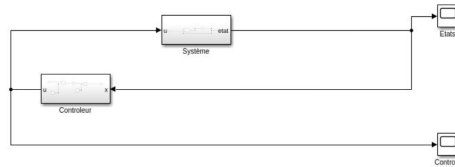


Figure 20:

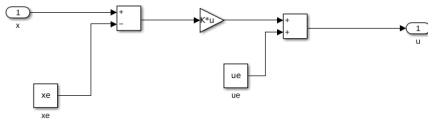


Figure 21: Contrôleur

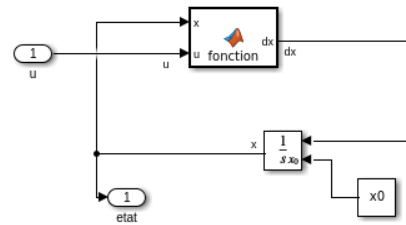


Figure 22: Système

Données à tester :

Cas	x_0	t_f	K	Intégrateur
Cas 1.1	$(0, 0, 0, 0)$	2	K	ode45
Cas 1.2	$(0, \pi/5, 0, 0)$	5	K	ode45
Cas 1.3	$(0, \pi/10, 0, 0)$	5	K	ode45

Figure 23: Tableau des données

- Cas 1.1

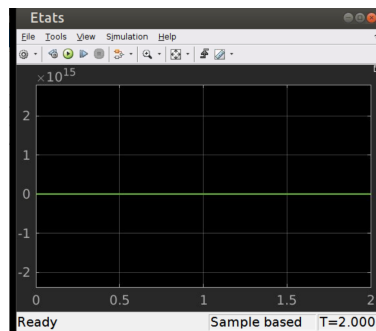


Figure 24: Simulation état Cas1.1

- Cas 1.2

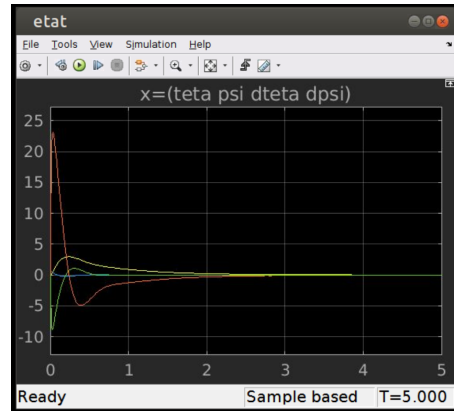


Figure 25: Simulation état Cas1.2

- Cas 1.3

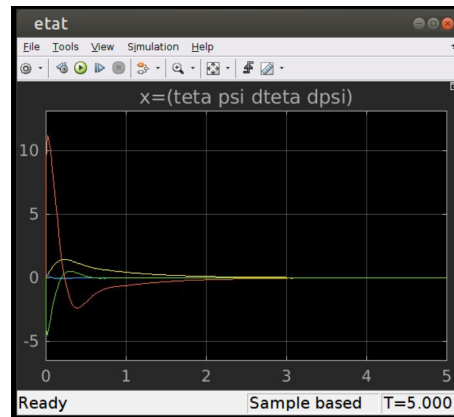


Figure 26: Simulation état Cas1.3

Analyse des resultats:

- On observe qu'en partant de l'origine, il n'y a aucune évolution d'aucun paramètre, ce qui montre que l'origine est un point d'équilibre stable pour le robot (cas 1.1).
- Pour les cas 1.2 et 1.3, seul la valeur de l'angle ψ de départ change (elle est divisée par deux, ce qui signifie qu'on incline deux fois moins la tête du robot). On remarque que les paramètres évoluent de manière identique, mais prennent des valeurs plus faibles dans le deuxième cas (tête moins inclinée implique moins grandes variations pour les angles). Cependant la position d'équilibre semble être atteinte au bout du même temps puisque dans les deux cas les courbes convergent au bout d'environ 3 secondes.

3.2 Introduction des capteurs et actionneurs

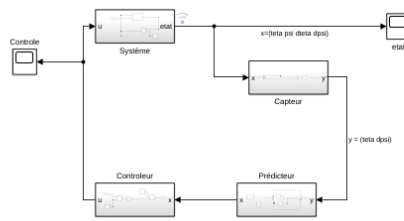


Figure 27:

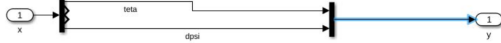


Figure 28: Capteur

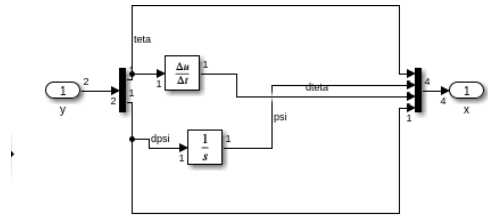


Figure 29: predicteur

Données à tester :

Cas	x_0	t_f	K	Intégrateur
Cas 1.1	$(0, 0, 0, 0)$	2	K	ode45
Cas 1.2	$(0, \pi/5, 0, 0)$	5	K	ode45
Cas 1.3	$(0, \pi/10, 0, 0)$	5	K	ode45

Figure 30: Tableau des données

- Cas 1.1

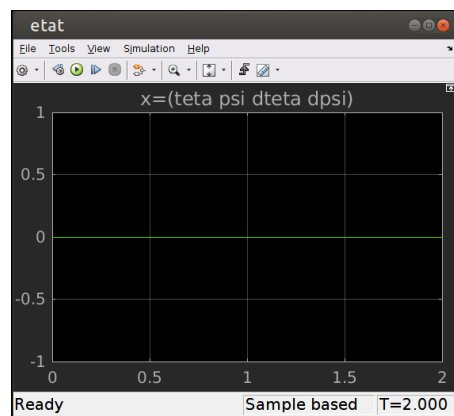


Figure 31: Simulation état Cas1.1

- Cas 1.2

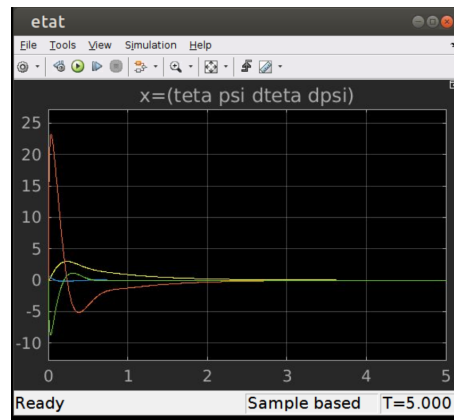


Figure 32: Simulation état Cas1.2

- Cas 1.3

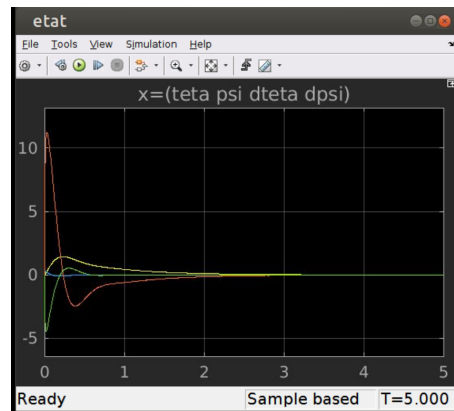


Figure 33: Simulation état Cas1.3

Analyse des resultats:

- L'origine est toujours un point d'équilibre stable pour le robot (aucune évolution d'aucun paramètre).
- On observe qu'on obtient les mêmes courbes que sans le predicteur et capteur. Il y a donc peut-être une erreur dans la définition de ces derniers.

3.3 Construction du modèle hybride

On décide d'effectuer une simulation en utilisant des opérateurs discrets (modification également de l'intégrateur dans le système avec un modèle discret) :



Figure 34: Capteur

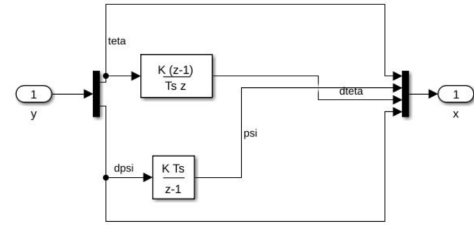


Figure 35: predicateur

On choisit d'effectuer les trois mêmes tests de données que précédemment :

- Cas 1.1



Figure 36: Simulation état 1er cas

- Cas 1.2

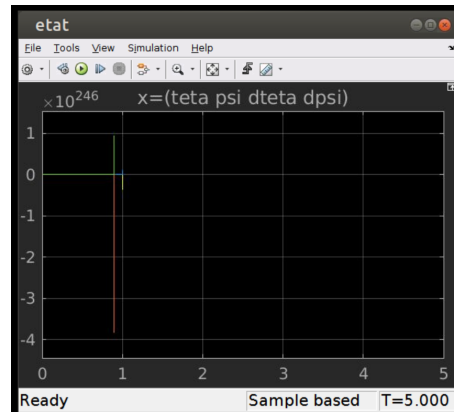


Figure 37: Simulation état Cas1.2

- Cas 1.3

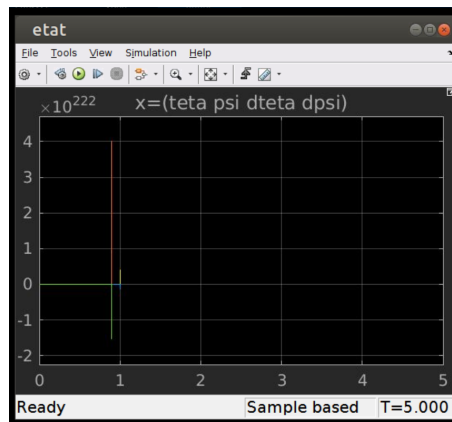


Figure 38: Simulation état Cas1.3

Analyse des resultats:

- Encore une fois, l'origine est point d'équilibre stable pour le robot.
- Les simulations des cas 1.2 et 1.3 ne sont pas abouties mais Simulink ne renvoie pas de message d'erreur pour autant. On ne peut pas conclure vis à vis de l'utilisation d'un modèle discret plutôt que continu.

4 Pendule inversé sur robot lego NXT -Conclusion

Après implémentation en C du code déduit à partir de cette étude sur matlab, on a pu tester l'équilibre du robot et se rendre compte qu'effectivement il parvient à retrouver sa position d'équilibre initiale après une légère perturbation. Mais si celle-ci est trop importante ou que le robot est mal calibré dès le départ alors il fuit en avançant/reculant avant de finalement tomber.